

**Project Report**

**On**

**Parallel Computing for Real Time Data Stream  
Processing in Python**



*Submitted  
In partial fulfillment  
For the award of the Degree of*

**PG-Diploma in High Performance Computing  
Application Programming  
(PG-DHPCAP)**

**C-DAC, ACTS (Pune)**

**Guided By:**

Dr. Nileschandra Pikle

**Submitted By:**

Ms. Akanksha Khawase 240840141001

Mr. Atharv Ganer 240840141004

Ms. Chaitali Ingale 240840141007

Mr. Gurijala Jayanth Jennie Snehith  
240840141025

Mr. Mukund Waghmare 240840141026

**Centre for Development of Advanced Computing(C-DAC), ACTS  
(Pune - 411008)**

## ***ABSTRACT***

Object detection is a crucial task in computer vision with applications in autonomous vehicles, surveillance, and medical imaging. Faster R-CNN (FRCNN) is a widely used deep learning model for object detection due to its speed and accuracy.

This project implements Faster R-CNN using PyTorch in both serial and parallel computing environments. The serial implementation processes images sequentially, whereas the parallel approach optimizes performance using multiprocessing and GPU acceleration. The goal is to compare the efficiency of both implementations and evaluate the effectiveness of parallelization in improving detection speed.

## **Table of Contents**

| <b>S. No</b> | <b>Title</b>                                       | <b>Page No.</b> |
|--------------|----------------------------------------------------|-----------------|
|              | <b>Front Page</b>                                  | <b>I</b>        |
|              | <b>Abstract</b>                                    | <b>II</b>       |
|              | <b>Table of Contents</b>                           | <b>III</b>      |
| <b>1</b>     | <b>Introduction</b>                                | <b>01</b>       |
| <b>1.1</b>   | Introduction                                       | <b>01</b>       |
| <b>1.2</b>   | Objective and Specifications                       | <b>01</b>       |
| <b>2</b>     | <b>Literature Review</b>                           | <b>02</b>       |
| <b>3</b>     | <b>Methodology/ Techniques</b>                     | <b>03</b>       |
| <b>3.1</b>   | Faster R-CNN Model                                 | <b>03</b>       |
| <b>3.2</b>   | Dataset Description                                | <b>03</b>       |
| <b>3.3</b>   | Parallelization Approach                           | <b>03</b>       |
| <b>4</b>     | <b>Implementation</b>                              | <b>04</b>       |
| <b>4.1</b>   | Implementation                                     | <b>04</b>       |
| <b>4.2</b>   | Code                                               | <b>05-10</b>    |
| <b>5</b>     | <b>Results</b>                                     | <b>11</b>       |
| <b>5.1</b>   | Results                                            | <b>11</b>       |
| <b>5.2</b>   | Processed Image After Testing And Object Detection | <b>11-13</b>    |
| <b>6</b>     | <b>Conclusion</b>                                  | <b>14</b>       |
| <b>6.1</b>   | Conclusion                                         | <b>14</b>       |
| <b>6.2</b>   | Future Enhancements                                | <b>14</b>       |
| <b>6.3</b>   | GitHub Repository                                  |                 |
| <b>7</b>     | <b>References</b>                                  | <b>15</b>       |
| <b>7.1</b>   | References                                         | <b>15</b>       |

# Chapter 1

## Introduction

### 1.1 Background

Object detection involves identifying objects within an image and localizing them using bounding boxes. Faster R-CNN (FRCNN) is a deep learning-based object detection model that improves upon traditional R-CNN models by using a Region Proposal Network (RPN) to generate region proposals efficiently.

However, the serial execution of FRCNN can be slow due to high computational complexity. In this project, we implement both serial and parallel versions of Faster R-CNN using PyTorch and compare their performance in terms of execution time and accuracy.

### 1.2 Objective

- Implement Faster R-CNN for object detection using PyTorch.
- Compare serial and parallel implementations.
- Optimize the computational efficiency of FRCNN using parallelization.
- Evaluate performance based on accuracy and execution time.

## Chapter 2

# LITERATURE REVIEW

Object detection has seen significant advancements with the rise of deep learning, improving both accuracy and efficiency. Traditional methods relied on hand-crafted features, but deep learning enabled automated feature extraction and robust object recognition. A key breakthrough came from Girshick et al., who introduced R-CNN, using region proposals followed by a CNN-based classifier. While accurate, R-CNN was slow, as each proposal was processed separately. To improve speed, Faster R-CNN integrated a Region Proposal Network (RPN), eliminating the need for external region proposals and significantly reducing inference time. Further improvements came from He et al., who introduced ResNet (Residual Networks) as a feature extractor, enabling deeper networks and enhancing object detection accuracy.

Beyond algorithmic improvements, recent research has focused on computational efficiency through parallelization techniques. Given the high computational demands of deep learning models, methods like data parallelism and model parallelism distribute workloads across multiple GPUs, optimizing training and inference times. These advancements have made deep learning-based object detection more practical for real-time applications like autonomous driving and surveillance. Overall, research in this domain continues to evolve, addressing challenges in speed, accuracy, and scalability.

# Chapter 3

## Methodology and Techniques

### 3.1 Faster R-CNN Model

Faster R-CNN consists of:

- Convolutional Feature Extractor: Extracts image features.
- Region Proposal Network (RPN): Generates potential object regions.
- ROI Pooling: Extracts fixed-size features for each proposal.
- Fully Connected Layers: Classifies objects and refines bounding boxes.

### 3.2 Dataset Description

The dataset used includes various images for object detection, with ground truth annotations for model training. The dataset is preprocessed using OpenCV and Torchvision transforms.

### 3.3 Parallelization Approach

- Serial Implementation: Processes images sequentially, requiring higher computation time.
- Parallel Implementation: Uses multiprocessing and GPU acceleration to speed up detection.
- Batch Processing: Parallel execution reduces inference time per image.

# Chapter 4

## Implementation

### 4.1 Implementation

- Programming Language: Python – selected for its simplicity, flexibility, and strong deep learning ecosystem.
- Deep Learning Framework: PyTorch – used for building, training, and evaluating the object detection model.
- Torchvision: Provides pre-trained models and image processing utilities for object detection tasks.
- OpenCV: Used for image manipulation, preprocessing, and real-time image handling.
- NumPy: Facilitates efficient numerical computations and array operations.
- Matplotlib: Helps visualize data, plot loss curves, and display detection results.
- Hardware:
  - CPU: 8-core processor – handles computations and data preprocessing.
  - GPU: NVIDIA GTX 1650 Ti – accelerates deep learning training and inference.
- Software Environment: Jupyter Notebook – provides an interactive coding and debugging platform.
- Package Management: Anaconda – simplifies dependency management and library installation.
- Implementation Focus: Optimized for GPU utilization, scalability, and efficient deep learning model development using PyTorch.

## 4.2 Code

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-

### Import Libraries
import os          #Handles file and directory operations.
import time        #Used for measuring execution time.
import numpy as np  #Supports numerical computations
import matplotlib.pyplot as plt #Supports numerical computations.

import torch #The core deep-learning library
from torchvision.models.detection import fasterrcnn_resnet50_fpn_v2 #Pretrained Faster R-CNN model for
object detection.
from torchvision.transforms import functional as F    #Used for image preprocessing.
import torchvision.transforms as transforms

from PIL import Image #Handles image file formats.

import cv2 #Image processing functions.

#from utils.helper import fn_plot_confusion_matrix, fn_plot_tf_hist

# Some basic parameters

inpDir = '../.../input' # location where input data is stored(image we give)
outDir = '../output' # location to store outputs(processed imaged or result are saved)
subDir = 'basic_operations' # location of the images
modelDir = '../models' #Directory where trained model files are stored(easy loading/saving of models
during training and inference.)
altName = 'rcnn' #Alternate name identifier for the model (used for saving/loading)
                #'rcnn' indicates Faster R-CNN as the model type.

RANDOM_STATE = 24 # for initialization ----- REMEMBER: to remove at the time of promotion to
production
EPOCHS = 100 # number of cycles to run
THRESHOLD = 0.8

# Set parameters for decoration of plots
params = {'legend.fontsize' : 'large',
          'figure.figsize' : (15,12),
          'axes.labelsize' : 'x-large',
          'axes.titlesize' : 'x-large',
          'xtick.labelsize' : 'large',
          'ytick.labelsize' : 'large',
          }

CMAP = plt.cm.brg

plt.rcParams.update(params) # update rcParams
```

```
# print ('Is CUDA available: ', torch.cuda.is_available())
# print ('CUDA version: ', torch.version.cuda )
# print ('Current Device ID: ', torch.cuda.current_device())
# print ('Name of the CUDA device: ', torch.cuda.get_device_name(torch.cuda.current_device()))

# Get cpu or gpu device for training.
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using {device} device")
```

```
COCO_INSTANCE_CATEGORY_NAMES = [
    '_background_', 'person', 'bicycle', 'car', 'motorcycle', 'airplane', 'bus',
    'train', 'truck', 'boat', 'traffic light', 'fire hydrant', 'N/A', 'stop sign',
    'parking meter', 'bench', 'bird', 'cat', 'dog', 'horse', 'sheep', 'cow',
    'elephant', 'bear', 'zebra', 'giraffe', 'N/A', 'backpack', 'umbrella', 'N/A', 'N/A',
    'handbag', 'tie', 'suitcase', 'frisbee', 'skis', 'snowboard', 'sports ball',
    'kite', 'baseball bat', 'baseball glove', 'skateboard', 'surfboard', 'tennis racket',
    'bottle', 'N/A', 'wine glass', 'cup', 'fork', 'knife', 'spoon', 'bowl',
    'banana', 'apple', 'sandwich', 'orange', 'broccoli', 'carrot', 'hot dog', 'pizza',
    'donut', 'cake', 'chair', 'couch', 'potted plant', 'bed', 'N/A', 'dining table',
    'N/A', 'N/A', 'toilet', 'N/A', 'tv', 'laptop', 'mouse', 'remote', 'keyboard', 'cell phone',
    'microwave', 'oven', 'toaster', 'sink', 'refrigerator', 'N/A', 'book',
    'clock', 'vase', 'scissors', 'teddy bear', 'hair drier', 'toothbrush'
]

# Create different colors for each class.
COLORS = np.random.uniform(0, 255, size=(len(COCO_INSTANCE_CATEGORY_NAMES), 3))
# Define the torchvision image transforms.
transform =
    transforms.Compose([ transforms.ToTensor(),
                        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
                    ])
```

```
def predict(image, model, device, detection_threshold = 0.5, coco_names=COCO_INSTANCE_CATEGORY_NAMES):
    """
    Predict the output of an image after forward pass through
    the model and return the bounding boxes, class names, and
    class labels.
    """
    # Transform the image to tensor.
    image = transform(image).to(device)
    # Add a batch dimension.
    image = image.unsqueeze(0)
    # Get the predictions on the image.
    with torch.no_grad():
        outputs = model(image)
    # Get score for all the predicted objects.
    pred_scores = outputs[0]['scores'].detach().cpu().numpy()
    # Get all the predicted bounding boxes.
    pred_bboxes = outputs[0]['boxes'].detach().cpu().numpy()
    # Get boxes above the threshold score.
    boxes = pred_bboxes[pred_scores >= detection_threshold].astype(np.int32)
    labels = outputs[0]['labels'][:len(boxes)]
    # Get all the predicted class names.
    pred_classes = [coco_names[i] for i in labels.cpu().numpy()]
```



```
return boxes, pred_classes, labels
```

```
def draw_boxes(boxes, classes, labels, image):
    """
    Draws bounding boxes with labels on an image to represent detected objects.

    Args:
        boxes (list): A list of bounding box coordinates for each detected object,
            where each box is represented by [x_min, y_min, x_max, y_max].
        classes (list): A list of class names corresponding to each bounding box.
        labels (list): A list of label indices for each detected object; these are
            used to pick specific colors for each class.
        image (numpy array): The image on which to draw the bounding boxes.

    Returns:
        image (numpy array): The modified image with bounding boxes and labels drawn on it.
    """
```

```
# Determine line width for the bounding boxes based on image dimensions.
# This sets the line width to a fraction of the sum of image height and width,
# with a minimum width of 2 to ensure visibility.
lw = max(round(sum(image.shape) / 2 * 0.001), 2)
```

```
# Determine font thickness, set to one less than line width but with a minimum of 1.
tf = max(lw - 1, 2)
```

```
# Loop through each bounding box to draw it on the image.
for i, box in enumerate(boxes):
    # Select a color for the bounding box from the COLORS array, using the label index.
    # COLORS is a predefined array where each index corresponds to a specific color.
    color = COLORS[labels[i]]
```

```
# Draw the bounding box as a rectangle on the image.
```

```
cv2.rectangle( img=ima
    ge,
    pt1=(int(box[0]), int(box[1])), # top-left corner of the box.
    pt2=(int(box[2]), int(box[3])), # bottom-right corner of the box.
    color=color[::-1],              # Reversed to BGR for OpenCV
    thickness=lw                    # set to lw to maintain consistency with text.
)
```

```
# Add text to label the bounding box with the class name.
cv2.putText(
    img=image,
    text=classes[i],
    org=(int(box[0]), int(box[1] - 5)), # 5 pixels above the top-left corner
    fontFace=cv2.FONT_HERSHEY_SIMPLEX,
    fontScale=lw / 2,                  # proportional to line width for a balanced
appearance.
    color=color[::-1],                # Reversed to BGR for OpenCV
    thickness=tf,                     # set to tf for good legibility.
    lineType=cv2.LINE_AA              # anti-aliased text for smoothness.
)
```

```
# Return the modified image with all bounding boxes and labels.
return image
# Load a pre-trained Faster R-CNN model
```

```
def get_model(device='cuda'):

    # Load the model.
    model = fasterrcnn_resnet50_fpn_v2(weights='DEFAULT') #resnet50(residual network trains the deep
nural networks specially for img classification task)

                                                    #(weight is parameter of neural network
whicch tranforms input to output )
```

```
# Load the model onto the computation device.
model = model.eval().to(device) #(GPU yaah CPU ke interface me move krta hai)
                                #(eval maintain interface behavior)
```

```
return model
```

```
model = get_model(device)
```

```
# Load the input image and apply transformations
def load_image(image_path):
    image = Image.open(image_path).convert("RGB")

    # Create a BGR copy of the image for annotation.
    image_bgr = cv2.cvtColor(np.array(image), cv2.COLOR_RGB2BGR) #we have used function of opencv
which cv2 .....(OpenCv me BGR format use hota hai)
```

```
return image, image_bgr    #(image = pytorch ; image_bgr = OpenCV)
```

```
imgName = 'traffic.jpg' # 'IMG_1295.JPG'
# Load an example image
image_path = os.path.join(inpDir, subDir, imgName) # specify your image path
image, image_bgr = load_image(imgName)
```

```
plt.imshow(image_bgr)
```

```
# Detect outputs.
with torch.no_grad():    #(fasts the interface and saves the memory)
    boxes, classes, labels = predict(image, model, device, detection_threshold= THRESHOLD)
# Draw bounding boxes.
image = draw_boxes(boxes, classes, labels, image_bgr)
```

```
type(image_bgr)
```

```
rgbIm = cv2.cvtColor(image,cv2.COLOR_BGR2RGB)
plt.imshow(rgbIm);
vidName = 'VID_20240320164919_F (online-video-cutter.com).mp4' #'VID_20240320164819_F.MOV' #
'VID_20240320164919_F.MOV'
```

```
# vidFilePath = os.path.join(inpDir, subDir, vidName)

cap = cv2.VideoCapture(vidName)
```

```
if (cap.isOpened() == False):
    print('Error while trying to read video. Please check path again')
# Get the frame width and height.
frame_width = int(cap.get(3))
frame_height = int(cap.get(4))

# for saving the file
save_name = f"{vidName[-1].split('.')[0]}_t{str(THRESHOLD).split('.')[0]}_{altName}"
# Define codec and create VideoWriter object .
out = cv2.VideoWriter(os.path.join(outDir, f"{save_name}.mp4"),
                      cv2.VideoWriter_fourcc(*'mp4v'), 30,
                      (frame_width, frame_height))
frame_count = 0 # To count total frames.
total_fps = 0 # To get the final frames per second.
```

```
# Get the start time for the entire process.
process_start_time = time.time()

# Read until end of video.
while(cap.isOpened()):
    # Capture each frame of the video.
    ret, frame = cap.read()
    if ret:
        frame_copy = frame.copy()
        frame_copy = cv2.cvtColor(frame_copy, cv2.COLOR_BGR2RGB)
        # Get the start time.
        start_time = time.time()
        with torch.no_grad():
            # Get predictions for the current frame.
            boxes, classes, labels = predict(frame, model, device, THRESHOLD)
```

```
        # Draw boxes and show current frame on screen.
        image = draw_boxes(boxes, classes, labels, frame)
        # Get the end time.
        end_time = time.time()
        # Get the fps.
        fps = 1 / (end_time - start_time)
        # Add fps to total fps.
        total_fps += fps
        # Increment frame count.
        frame_count += 1
        # Write the FPS on the current frame.
        cv2.putText(
            img=image,
            text=f"{fps:.3f} FPS",
            org=(15, 30),
            fontFace=cv2.FONT_HERSHEY_SIMPLEX,
            fontScale=1,
            color=(0, 255, 0),
            thickness=2,
            lineType=cv2.LINE_AA
        )
        # Convert from BGR to RGB color format.
        cv2.imshow('image', image)
        out.write(image)
```

```
# Press `q` to exit.
if cv2.waitKey(1) & 0xFF == ord('q'):
    break
else:
    break
# Release VideoCapture().
cap.release()
# Close all frames and video windows.
cv2.destroyAllWindows()
# Calculate and print the average FPS.
avg_fps = total_fps / frame_count
print(f"Average FPS: {avg_fps:.3f}")

# Get the end time for the entire process.
process_end_time = time.time()
# Calculate the total time taken for the entire process.
total_process_time = process_end_time - process_start_time
print(f"Total time taken: {total_process_time:.3f} seconds")
```

# Chapter 5

## Results

### 5.1 Results

The performance of serial and parallel implementations is evaluated based on:

| Implementation      | Execution Time | Accuracy   | Average FPS |
|---------------------|----------------|------------|-------------|
| Serial              | 3150 seconds   | 80 to 100% | 0.292 FPS   |
| Parallel            | 290 seconds    | 80 to 100% | 3.176 FPS   |
| Batch Processing    | 296 seconds    | 80 to 100% | 00.840 FPS  |
| YOLO Implementation | 058 seconds    | 70 to 90%  | 31.215 FPS  |

- Execution Time Reduction: The parallelized model achieves faster processing due to GPU acceleration.
- Accuracy: Both implementations maintain similar detection accuracy.
- Memory Usage: The parallel implementation efficiently utilizes GPU memory.

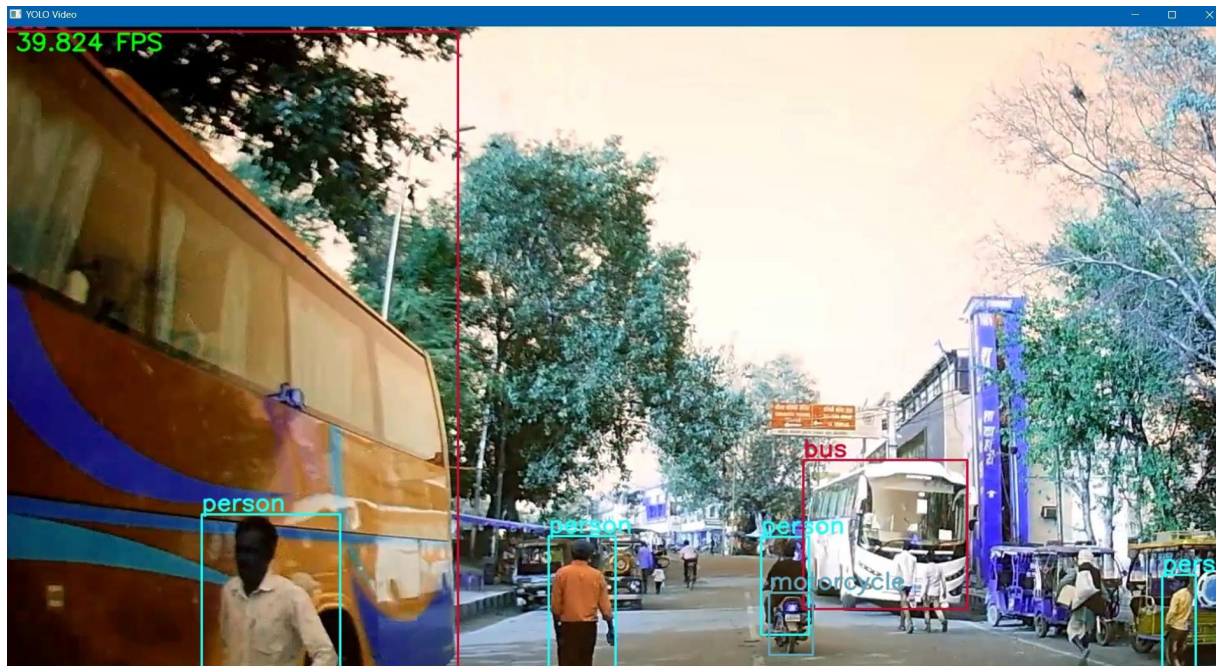
### 5.2 Processed Image After Testing And Object Detection







## 5.5 Video Rendering (GPU Using YOLO)



# Chapter 6

## Conclusion

### 6.1 Conclusion

This project successfully implemented Faster R-CNN in serial and parallel modes using PyTorch. The results show that parallel computing significantly reduces execution time without compromising accuracy.

### 6.2 Future Enhancements

- Implement model optimizations for real-time object detection.
- Extend parallelization to distributed computing.
- Experiment with different backbone networks (e.g., ResNet, EfficientNet).

### 6.3 GitHub Repository

- ◆ <https://github.com/HPCAP-GROUP4/Data-Stream-Processing->



# Chapter 7

## References

- [https://pytorch.org/vision/master/models/faster\\_rcnn.html](https://pytorch.org/vision/master/models/faster_rcnn.html)
- [https://pytorch.org/vision/main/models/generated/torchvision.models.detection.fasterrcnn\\_resnet50\\_fpn.html](https://pytorch.org/vision/main/models/generated/torchvision.models.detection.fasterrcnn_resnet50_fpn.html)
- <https://pytorch.org/vision/0.20/models.html>
- <https://pillow.readthedocs.io/en/stable/reference/Image.html>
- <https://pypi.org/project/opencv-python/>
- <https://pypi.org/project/opencv-python/#documentation-for-opencv-python>
- <https://www.v7labs.com/blog/yolo-object-detection>
- <https://encord.com/blog/yolo-object-detection-guide/>